

Aula 11 — Comunicação com Web API

Módulo: Blazor

Carga horária: 4 horas

Projetos base:

- **API:** Aula11.Api (ASP.NET Core)
- **Front:** Aula10.BlazorWasm OU Aula08.BlazorServer

Objetivo da aula:

Dominar **REST**, **DTOs**, **HttpClient** e **versionamento de API**, implementando **paginação e filtros** no backend e consumindo corretamente no **Blazor**.

PARTE 1 — TEORIA (≈ 2 horas)

1 REST (Representational State Transfer)

◆ Princípios

- **Recursos (URI):** /api/produtos
- **Verbos HTTP:**
 - GET → leitura
 - POST → criação
 - PUT/PATCH → atualização
 - DELETE → remoção
- **Stateless:** cada request é independente
- **Status Codes:**
 - 200 OK , 201 Created
 - 400 Bad Request
 - 404 Not Found

- 500 Internal Server Error

👉 Boa prática: API não retorna View, apenas JSON.

2 DTOs (Data Transfer Objects)

◆ Por que usar?

- Não expor entidades internas
- Versionamento mais seguro
- Contrato claro entre API ↔ Front

```
public record ProdutoDto(int Id, string Nome, decimal Preco);
```

3 HttpClient

◆ Papel

- Cliente HTTP padrão do .NET
- Async por padrão
- Integração nativa com JSON

```
await http.GetFromJsonAsync<List<ProdutoDto>>("api/v1/produtos");
```

📌 Regra de ouro:

Nunca instanciar `new HttpClient()` diretamente no código de página — usar DI.

Versionamento de API

◆ Por que versionar?

- Evoluir sem quebrar clientes
- Suporte a múltiplas versões

◆ Estratégias

- URL: /api/v1/produtos
- Header: api-version
- QueryString: ?api-version=1.0

👉 Nesta aula: versionamento por URL.

PARTE 2 — PRÁTICA (≈ 2 horas)

Objetivo Prático

Criar uma API REST versionada, com paginação e filtros, e consumir no Blazor usando HttpClient.

Criando a API ASP.NET Core

```
dotnet new webapi -n Aula11.Api cd Aula11.Api
```

Model (Entidade)

 Models/Produto.cs

```
namespace Aula11.Api.Models { public class Produto { public int Id { get; set; } public string Nome { get; set; } public decimal  
Preco { get; set; } public bool Ativo { get; set; } } }
```

3 DTOs

 DTOs/ProdutoDto.cs

```
namespace Aula11.Api.DTOs { public record ProdutoDto(int Id, string Nome, decimal Preco); }
```

4 Resultado Paginado (Wrapper)

 DTOs/PagedResult.cs

```
namespace Aula11.Api.DTOs { public class PagedResult<T> { public int Page { get; set; } public int PageSize { get; set; } public  
int TotalItems { get; set; } public List<T> Items { get; set; } } }
```

5 Controller REST Versionado

 Controllers/V1/ProdutosController.cs

```
using Aula11.Api.DTOs; using Aula11.Api.Models; using Microsoft.AspNetCore.Mvc; namespace Aula11.Api.Controllers.V1 {  
[ApiController] [Route("api/v1/[controller]")] public class ProdutosController : ControllerBase { private static List<Produto>  
_produtos = new() { new Produto { Id = 1, Nome = "Notebook", Preco = 4500, Ativo = true }, new Produto { Id = 2, Nome = "Mouse",  
Preco = 150, Ativo = true }, new Produto { Id = 3, Nome = "Teclado", Preco = 300, Ativo = false } }; // GET api/v1/produtos?  
page=1&pageSize=2&ativo=true&nome=note [HttpGet] public IActionResult Get( int page = 1, int pageSize = 10, bool? ativo = null,  
string? nome = null) { var query = _produtos.AsQueryable(); // Filtro por status if (ativo.HasValue) query = query.Where(p =>  
p.Ativo == ativo.Value); // Filtro por nome if (!string.IsNullOrEmpty(nome)) query = query.Where(p => p.Nome.Contains(nome,
```

```
StringComparison.OrdinalIgnoreCase)); var total = query.Count(); var items = query .Skip((page - 1) * pageSize) .Take(pageSize)
.Select(p => new ProdutoDto(p.Id, p.Nome, p.Preco)) .ToList(); var result = new PagedResult<ProdutoDto> { Page = page, PageSize =
pageSize, TotalItems = total, Items = items }; return Ok(result); } } }
```

- ✓ REST puro
 - ✓ DTOs
 - ✓ Paginação
 - ✓ Filtros dinâmicos
-

6 Consumindo a API no Blazor

DTOs no Front

 Models/PagedResult.cs

```
public class PagedResult<T> { public int Page { get; set; } public int PageSize { get; set; } public int TotalItems { get; set; }
public List<T> Items { get; set; } }
```

 Models/ProdutoDto.cs

```
public record ProdutoDto(int Id, string Nome, decimal Preco);
```

7 Página Blazor com Filtros e Paginação

 Pages/Produtos.razor

```
@page "/produtos" @inject HttpClient Http <h3>Produtos</h3> <input placeholder="Nome" @bind="filtroNome" /> <select
@bind="filtroAtivo"> <option value="">Todos</option> <option value="true">Ativos</option> <option value="false">Inativos</option>
</select> <button @onclick="Buscar">Buscar</button> @if (resultado == null) { <p>Carregando...</p> } else { <ul> @foreach (var p
```

```
in resultado.Items) { <li>@p.Nome - @p.Preco.ToString("C")</li> } </ul> <button @onclick="Anterior" disabled="@((page == 1)">Anterior</button> <button @onclick="Proxima">Próxima</button> } @code { PagedResult<ProdutoDto>? resultado; int page = 1; int pageSize = 2; string? filtroNome; bool? filtroAtivo; protected override async Task OnInitializedAsync() { await Buscar(); } async Task Buscar() { var url = $"api/v1/produtos?page={page}&pageSize={pageSize}"; if (!string.IsNullOrEmpty(filtroNome)) url += $"&nome={filtroNome}"; if (filtroAtivo.HasValue) url += $"&ativo={filtroAtivo.Value}"; resultado = await Http.GetFromJsonAsync<PagedResult<ProdutoDto>>(url); } async Task Proxima() { page++; await Buscar(); } async Task Anterior() { page--; await Buscar(); } }
```

- ✓ Consumo real
 - ✓ Filtros dinâmicos
 - ✓ Paginação funcional
-

✓ Resultado Esperado

- ✓ API REST versionada
 - ✓ DTOs bem definidos
 - ✓ Paginação e filtros
 - ✓ Blazor consumindo API corretamente
 - ✓ Base sólida para JWT e segurança
-

🚩 Atividade Avaliativa

👉 Desafio prático:

1. Criar POST /api/v1/produtos
2. Validar DTO no backend
3. Consumir POST no Blazor
4. Exibir mensagens de erro/sucesso
5. Criar api/v2/produtos com campo extra